```sql
-- Creating a fresh database for this task

create database ShopEase;
GO

use ShopEase;

-- ============================================================
-- TABLE: customers
-- Stores customer profile info
-- ============================================================
CREATE TABLE customers (
    customer_id  INT           PRIMARY KEY,
    first_name   VARCHAR(50)   NOT NULL,
    last_name    VARCHAR(50)   NOT NULL,
    email        VARCHAR(100)  UNIQUE NOT NULL,   -- no two customers can share an email
    city         VARCHAR(50)   NOT NULL,
    state        VARCHAR(50)   NOT NULL,
    join_date    DATE          NOT NULL,
    is_premium   BIT           DEFAULT 0          -- 0 = FALSE, 1 = TRUE (SSMS uses BIT for boolean)
);

-- Indexes to speed up city/state filters
CREATE INDEX idx_customers_city  ON customers(city);
CREATE INDEX idx_customers_state ON customers(state);
GO

-- ============================================================
-- TABLE: products
-- Stores product catalog with category, brand, price and stock
-- ============================================================
CREATE TABLE products (
    product_id   INT             PRIMARY KEY,
    product_name VARCHAR(100)    NOT NULL,
    category     VARCHAR(50)     NOT NULL,
    brand        VARCHAR(50)     NOT NULL,
    unit_price   DECIMAL(10,2)   NOT NULL CHECK (unit_price > 0),   -- price must be positive
    stock_qty    INT             NOT NULL DEFAULT 0 CHECK (stock_qty >= 0)
);

-- Index on category since many queries filter by category
CREATE INDEX idx_products_category ON products(category);
GO

-- ============================================================
-- TABLE: orders
```

```sql
-- Each row is one customer order; links back to customers
-- ============================================================
CREATE TABLE orders (
    order_id      INT           PRIMARY KEY,
    customer_id   INT           NOT NULL,
    order_date    DATE          NOT NULL,
    status        VARCHAR(20)   NOT NULL DEFAULT 'Pending'
                  CHECK (status IN
                    ('Pending','Shipped','Delivered','Cancelled')),
    total_amount  DECIMAL(12,2) NOT NULL CHECK (total_amount >= 0),

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Indexes help with date-range queries and status filters
CREATE INDEX idx_orders_date   ON orders(order_date);
CREATE INDEX idx_orders_status ON orders(status);
GO

-- ============================================================
-- TABLE: order_items
-- Line items inside each order (which product, how many, price)
-- ============================================================
CREATE TABLE order_items (
    item_id      INT           PRIMARY KEY,
    order_id     INT           NOT NULL,
    product_id   INT           NOT NULL,
    quantity     INT           NOT NULL CHECK (quantity > 0),
    unit_price   DECIMAL(10,2) NOT NULL CHECK (unit_price > 0),
    discount_pct DECIMAL(5,2)  DEFAULT 0 CHECK (discount_pct BETWEEN 0
      AND 100),

    FOREIGN KEY (order_id)   REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
GO

-- ========== INSERT: customers ==========
INSERT INTO customers VALUES
(101, 'Aarav',  'Sharma', 'aarav.s@email.com',  'Mumbai',
  'Maharashtra', '2024-01-15', 1),
(102, 'Priya',  'Patel',  'priya.p@email.com',  'Ahmedabad', '
  Gujarat',     '2024-02-20', 0),
(103, 'Rohan',  'Gupta',  'rohan.g@email.com',  'Delhi',     '
  Delhi',       '2024-03-10', 1),
(104, 'Sneha',  'Reddy',  'sneha.r@email.com',  'Hyderabad', '
  Telangana',   '2024-04-05', 0),
(105, 'Vikram', 'Singh',  'vikram.s@email.com', 'Jaipur',    '
  Rajasthan',   '2024-05-12', 1),
```

```
 89  (106, 'Ananya', 'Iyer',   'ananya.i@email.com', 'Chennai',   'Tamil
       Nadu',  '2024-06-18', 0),
 90  (107, 'Karan',  'Mehta',  'karan.m@email.com',  'Pune',
       'Maharashtra', '2024-07-22', 1),
 91  (108, 'Divya',  'Nair',   'divya.n@email.com',  'Kochi',     '
       Kerala',      '2024-08-30', 0);
 92
 93  -- ========== INSERT: products ==========
 94  INSERT INTO products VALUES
 95  (201, 'Wireless Earbuds',      'Electronics', 'BoAt',         1499.00,
       250),
 96  (202, 'Cotton T-Shirt',        'Clothing',    'Levis',         799.00,
       500),
 97  (203, 'Smart Watch',           'Electronics', 'Noise',        2999.00,
       150),
 98  (204, 'Running Shoes',         'Clothing',    'Nike',         4599.00,
       120),
 99  (205, 'Bluetooth Speaker',     'Electronics', 'JBL',          3499.00,
       200),
100  (206, 'Bedsheet Set',          'Home',        'Spaces',       1299.00,
       300),
101  (207, 'Laptop Stand',          'Electronics', 'AmazonBasics',  899.00,
       180),
102  (208, 'Cushion Covers (Set)',  'Home',        'HomeCenter',    599.00,
       400);
103
104  -- ========== INSERT: orders ==========
105  INSERT INTO orders VALUES
106  (1001, 101, '2024-08-01', 'Delivered',  4498.00),
107  (1002, 102, '2024-08-03', 'Delivered',   799.00),
108  (1003, 103, '2024-08-05', 'Shipped',    7498.00),
109  (1004, 101, '2024-08-10', 'Delivered',  3499.00),
110  (1005, 104, '2024-08-12', 'Cancelled',  2999.00),
111  (1006, 105, '2024-08-15', 'Delivered',  5898.00),
112  (1007, 106, '2024-08-18', 'Pending',    1299.00),
113  (1008, 103, '2024-08-20', 'Delivered',   899.00),
114  (1009, 107, '2024-08-25', 'Shipped',    6098.00),
115  (1010, 108, '2024-08-28', 'Delivered',  1598.00);
116
117  -- ========== INSERT: order_items ==========
118  INSERT INTO order_items VALUES
119  (5001, 1001, 201, 2, 1499.00,  0),
120  (5002, 1001, 207, 1,  899.00, 10),
121  (5003, 1002, 202, 1,  799.00,  0),
122  (5004, 1003, 203, 1, 2999.00,  0),
123  (5005, 1003, 204, 1, 4599.00,  5),
124  (5006, 1004, 205, 1, 3499.00,  0),
125  (5007, 1005, 203, 1, 2999.00,  0),
126  (5008, 1006, 201, 1, 1499.00, 10),
```

```
(5009, 1006, 204, 1, 4599.00,  5),
(5010, 1007, 206, 1, 1299.00,  0),
(5011, 1008, 207, 1,  899.00,  0),
(5012, 1009, 205, 1, 3499.00,  0),
(5013, 1009, 208, 2,  599.00, 15),
(5014, 1010, 206, 1, 1299.00,  0),
(5015, 1010, 208, 1,  599.00,  0);
GO
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
------------------
--Section A – SQL Basics (SELECT, Constraints, Primary Keys)
-- Simple SELECT * to pull the full customer table
SELECT *
FROM customers;

-- Selecting only the three columns we need instead of pulling the full
row
SELECT first_name, last_name, city
FROM customers;

-- DISTINCT removes duplicate category values so each appears only once
SELECT DISTINCT category
FROM products;


email VARCHAR(100) UNIQUE NOT NULL

-- This will fail because aarav.s@email.com is already in the table
INSERT INTO customers VALUES
(109, 'Test', 'User', 'aarav.s@email.com', 'Mumbai', 'Maharashtra',
'2024-09-01', 0);


-- Attempting to insert a product with a negative price
-- The CHECK constraint on unit_price should block this
INSERT INTO products VALUES
(209, 'Broken Item', 'Electronics', 'NoName', -50.00, 100);


-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
-------------------------------------------------------------------------------
-----
--Section B – Filtering & Optimization (WHERE, Indexes)
-- Filter orders table to only show delivered orders
SELECT *
FROM orders
WHERE status = 'Delivered';
```

```sql

-- Combining two WHERE conditions with AND
SELECT product_id, product_name, brand, unit_price
FROM products
WHERE category = 'Electronics'
  AND unit_price > 2000;


  -- YEAR() extracts the year part from the join_date column
-- Then we also filter state
SELECT customer_id, first_name, last_name, city, join_date
FROM customers
WHERE YEAR(join_date) = 2024
  AND state = 'Maharashtra';

  -- BETWEEN is inclusive on both ends
-- Adding NOT IN to exclude cancelled orders
SELECT order_id, customer_id, order_date, status, total_amount
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
  AND status <> 'Cancelled';

CREATE INDEX idx_orders_date ON orders(order_date);

-- This query benefits directly from idx_orders_date
-- SQL Server uses an index seek instead of a table scan
SELECT *
FROM orders
WHERE order_date >= '2024-08-15'
  AND order_date <= '2024-08-31';

  SELECT * FROM customers WHERE YEAR(join_date) = 2024;

 -- Instead of applying a function on the column,
-- we express the same condition as a range on the raw date
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
  AND join_date <  '2025-01-01';


    ------------------------------------------------------------------------ ↵
    ------------------------------------------------------------

--Section C – Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)
-- COUNT(*) counts every row regardless of NULL values
SELECT COUNT(*) AS total_orders
FROM orders;

```

```sql
-- SUM adds up the total_amount for all rows matching the WHERE filter
SELECT SUM(total_amount) AS total_delivered_revenue
FROM orders
WHERE status = 'Delivered';

-- GROUP BY splits the table into groups, AVG() runs within each group
SELECT   category,
         ROUND(AVG(unit_price), 2) AS avg_price
FROM     products
GROUP BY category
ORDER BY avg_price DESC;

-- Grouping by status gives us one summary row per unique status value
SELECT   status,
         COUNT(*)          AS order_count,
         SUM(total_amount) AS total_revenue
FROM     orders
GROUP BY status
ORDER BY total_revenue DESC;


-- MAX and MIN work within each GROUP BY partition
SELECT   category,
         MAX(unit_price) AS most_expensive,
         MIN(unit_price) AS cheapest
FROM     products
GROUP BY category;


-- HAVING filters groups AFTER aggregation, unlike WHERE which filters rows before
-- We can't use WHERE AVG(...) - that's a syntax error in SQL
SELECT   category,
         ROUND(AVG(unit_price), 2) AS avg_price
FROM     products
GROUP BY category
HAVING   AVG(unit_price) > 2000;



-------------------------------------------------------------------------------
--Section D - Joins & Relationships
-- INNER JOIN returns only rows that have a match in BOTH tables
-- Customers with no orders and orders with no customer are both excluded
SELECT   o.order_id,
         o.order_date,
         c.first_name,
         c.last_name,
```

```sql
        o.total_amount
FROM    orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.customer_id
ORDER BY o.order_date;

-- LEFT JOIN keeps all rows from the LEFT table (customers)
-- If a customer has no matching order, order columns show NULL
SELECT   c.customer_id,
         c.first_name,
         c.last_name,
         o.order_id,
         o.order_date,
         o.status
FROM    customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id
ORDER BY c.customer_id;


-- Joining three tables: orders -> order_items -> products
-- This shows what was actually purchased in each order
SELECT   o.order_id,
         p.product_name,
         oi.quantity,
         oi.unit_price,
         oi.discount_pct
FROM    orders AS o
INNER JOIN order_items AS oi
    ON o.order_id = oi.order_id
INNER JOIN products AS p
    ON oi.product_id = p.product_id
ORDER BY o.order_id, p.product_name;

-- Example: all customers, with their orders if they have any
SELECT c.first_name, o.order_id
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;

-- Example: all orders, including any that somehow lack a valid customer
  record
-- (shouldn't happen with FK constraints, but useful to check)
SELECT c.first_name, o.order_id
FROM   customers c
RIGHT JOIN orders o ON c.customer_id = o.customer_id;


-- Useful when you want to see ALL customers AND ALL orders,
-- even if some don't have a counterpart
```

```sql
SELECT c.first_name, o.order_id
FROM   customers c
FULL OUTER JOIN orders o ON c.customer_id = o.customer_id;


-- customer_id 999 does not exist in the customers table
INSERT INTO orders VALUES (1099, 999, '2024-09-01', 'Pending', 500.00);

-------------------------------------------------------------------------------
  -------------------------------------------------------------------------------
  ---------------------------------------------------------------

--Section E – Advanced Concepts (CASE, ACID, Transactions)
-- CASE works like an if-else inside a SELECT statement
SELECT product_name,
       unit_price,
       CASE
           WHEN unit_price < 1000              THEN 'Budget'
           WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
           WHEN unit_price > 3000              THEN 'Premium'
       END AS price_tier
FROM products
ORDER BY unit_price;



-- Using CASE inside SUM() to conditionally count rows
-- SUM(CASE WHEN ... THEN 1 ELSE 0 END) is a classic pivot pattern
SELECT
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS
      delivered_count,
    SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS
      not_delivered_count
FROM orders;


-- This transaction does 4 things atomically:
-- 1. Inserts a new order for customer 102
-- 2. Inserts two line items for that order
-- 3. Updates stock quantities of both purchased products
-- 4. COMMITs if all steps succeed, otherwise ROLLBACKs everything

BEGIN TRY
    BEGIN TRANSACTION;

    -- Step 1: Insert the new order
    INSERT INTO orders (order_id, customer_id, order_date, status,
      total_amount)
    VALUES (1011, 102, CAST(GETDATE() AS DATE), 'Pending', 1598.00);
```

```sql

    -- Step 2a: First line item – 1 unit of Laptop Stand (product 207, ₹899)
    INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
    VALUES (5016, 1011, 207, 1, 899.00, 0);

    -- Step 2b: Second line item – 1 unit of Cushion Covers (product 208, ₹599)
    INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price, discount_pct)
    VALUES (5017, 1011, 208, 1, 599.00, 0);

    -- Step 3a: Reduce stock for Laptop Stand by 1
    UPDATE products
    SET    stock_qty = stock_qty - 1
    WHERE  product_id = 207;

    -- Step 3b: Reduce stock for Cushion Covers by 1
    UPDATE products
    SET    stock_qty = stock_qty - 1
    WHERE  product_id = 208;

    -- If we reached here without errors, commit everything
    COMMIT TRANSACTION;
    PRINT 'Transaction committed successfully. Order 1011 placed.';

END TRY
BEGIN CATCH
    -- If anything above failed, undo all changes
    ROLLBACK TRANSACTION;
    PRINT 'Transaction rolled back due to error: ' + ERROR_MESSAGE();
END CATCH;
GO

-- Verify the new order was inserted
SELECT * FROM orders WHERE order_id = 1011;

-- Verify stock was reduced for both products
SELECT product_id, product_name, stock_qty
FROM   products
WHERE  product_id IN (207, 208);


```